

MC714 - Sistemas Distribuidos

slides by Maarten van Steen
(adapted from Distributed System - 3rd Edition)

Chapter 05: Naming

Version: May 10, 2022

Naming

Essence

Names are used to denote entities in a distributed system. To operate on an entity, we need to access it at an **access point**. Access points are entities that are named by means of an **address**.

Note

A **location-independent** name for an entity E , is independent from the addresses of the access points offered by E .

Identifiers

Pure name

A name that has no meaning at all; it is just a random string. Pure names can be used for comparison only.

Identifier: A name having some specific properties

- 1 An identifier refers to at most one entity.
- 2 Each entity is referred to by at most one identifier.
- 3 An identifier always refers to the same entity (i.e., it is never reused).

Observation

An identifier need not necessarily be a pure name, i.e., it may have content.

Broadcasting

Broadcast the ID, requesting the entity to return its current address

- Can never scale beyond local-area networks
- Requires all processes to listen to incoming location requests

Address Resolution Protocol (ARP)

To find out which MAC address is associated with an IP address, broadcast the query “who has this IP address”?

Forwarding pointers

When an entity moves, it leaves behind a pointer to its next location

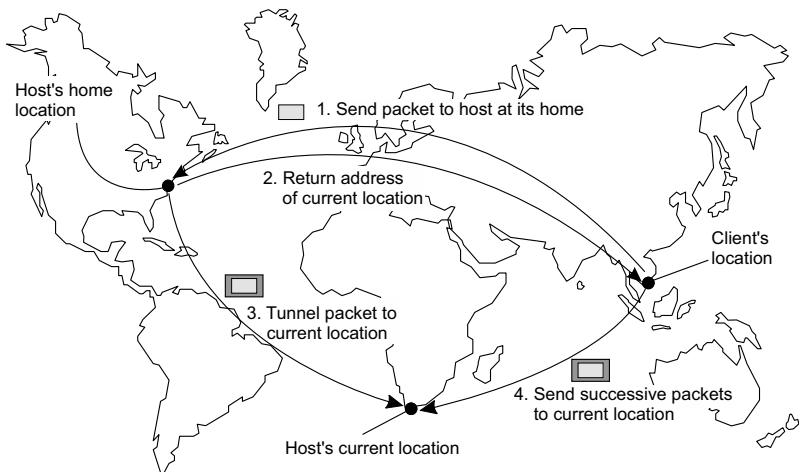
- Dereferencing can be made entirely transparent to clients by simply following the chain of pointers
- Update a client's reference when present location is found
- Geographical scalability problems (for which separate chain reduction mechanisms are needed):
 - Long chains are not fault tolerant
 - Increased network latency at dereferencing

Home-based approaches

Single-tiered scheme: Let a **home** keep track of where the entity is

- Entity's **home address** registered at a naming service
- The home registers the **foreign address** of the entity
- Client contacts the home first, and then continues with foreign location

The principle of mobile IP



Home-based approaches

Problems with home-based approaches

- Home address has to be supported for entity's lifetime
- Home address is fixed $\Rightarrow$ unnecessary burden when the entity permanently moves
- Poor geographical scalability (entity may be next to client)

Note

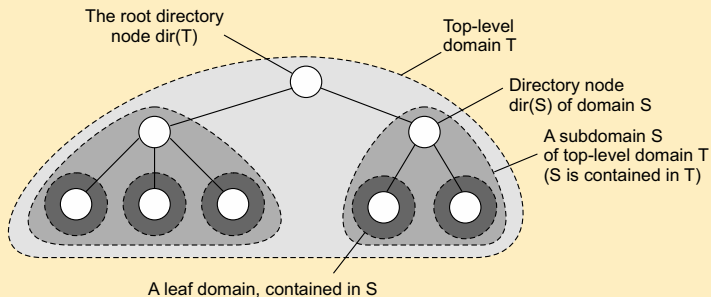
Permanent moves may be tackled with another level of naming (DNS)

Hierarchical Location Services (HLS)

Basic idea

Build a large-scale search tree for which the underlying network is divided into hierarchical domains. Each domain is represented by a separate directory node.

Principle

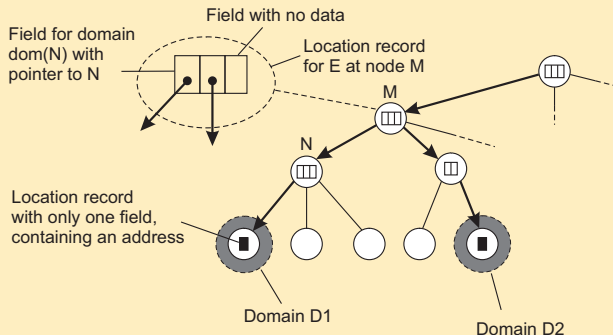


HLS: Tree organization

Invariants

- Address of entity E is stored in a leaf or intermediate node
- Intermediate nodes contain a pointer to a child if and only if the subtree rooted at the child stores an address of the entity
- The root knows about all entities

Storing information of an entity having two addresses in different leaf domains

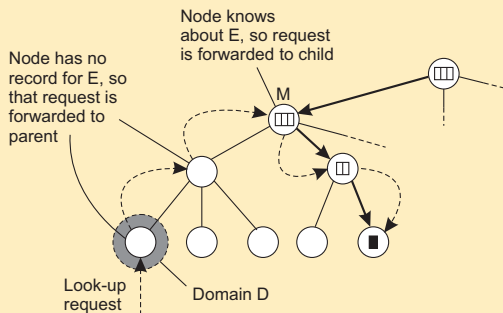


HLS: Lookup operation

Basic principles

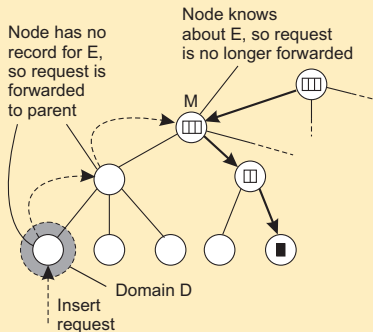
- Start lookup at local leaf node
- Node knows about $E \Rightarrow$ follow downward pointer, else go up
- Upward lookup always stops at root

Looking up a location

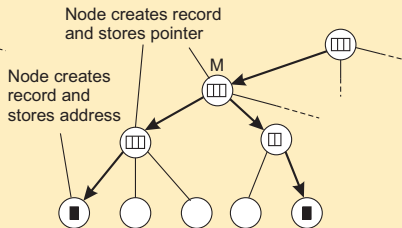


HLS: Insert operation

- An insert request is forwarded to the first node that knows about entity E .
- A chain of forwarding pointers to the leaf node is created



(a)



(b)

Can an HLS scale?

Observation

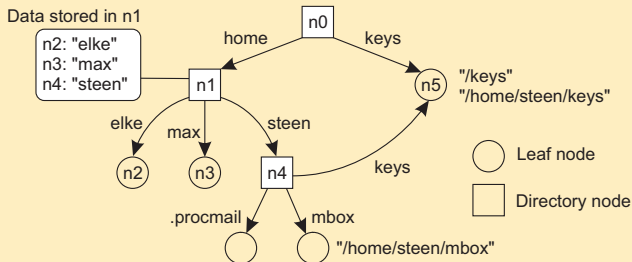
A design flaw seems to be that the root node needs to keep track of **all** identifiers $\Rightarrow$ make a distinction between a **logical design** and its **physical implementation**.

Name space

Naming graph

A graph in which a **leaf node** represents a (named) entity. A **directory node** is an entity that refers to other nodes.

A general naming graph with a single root node



Note

A directory node contains a table of *(node identifier, edge label)* pairs.

Name space

We can easily store all kinds of **attributes** in a node

- Type of the entity
- An identifier for that entity
- Address of the entity's location
- Nicknames
- ...

Name space

We can easily store all kinds of **attributes** in a node

- Type of the entity
- An identifier for that entity
- Address of the entity's location
- Nicknames
- ...

Note

Directory nodes can also have attributes, besides just storing a directory table with *(identifier, label)* pairs.

Name resolution

Problem

To resolve a name we need a directory node. How do we actually find that (initial) node?

Name resolution

Problem

To resolve a name we need a directory node. How do we actually find that (initial) node?

Closure mechanism: The mechanism to select the implicit context from which to start name resolution

- `www.distributed-systems.net`: start at a DNS name server
- `/home/maarten/mbox`: start at the local NFS file server (possible recursive search)
- `0031 20 598 7784`: dial a phone number
- `77.167.55.6`: route message to a specific IP address

Name resolution

Problem

To resolve a name we need a directory node. How do we actually find that (initial) node?

Closure mechanism: The mechanism to select the implicit context from which to start name resolution

- `www.distributed-systems.net`: start at a DNS name server
- `/home/maarten/mbox`: start at the local NFS file server (possible recursive search)
- `0031 20 598 7784`: dial a phone number
- `77.167.55.6`: route message to a specific IP address

Name-space implementation

Basic issue

Distribute the name resolution process as well as name space management across multiple machines, by distributing nodes of the naming graph.

Name-space implementation

Basic issue

Distribute the name resolution process as well as name space management across multiple machines, by distributing nodes of the naming graph.

Distinguish three levels

Name-space implementation

Basic issue

Distribute the name resolution process as well as name space management across multiple machines, by distributing nodes of the naming graph.

Distinguish three levels

- **Global level:** Consists of the high-level directory nodes. Main aspect is that these directory nodes have to be jointly managed by different administrations

Name-space implementation

Basic issue

Distribute the name resolution process as well as name space management across multiple machines, by distributing nodes of the naming graph.

Distinguish three levels

- **Global level:** Consists of the high-level directory nodes. Main aspect is that these directory nodes have to be jointly managed by different administrations
- **Administrational level:** Contains mid-level directory nodes that can be grouped in such a way that each group can be assigned to a separate administration.

Name-space implementation

Basic issue

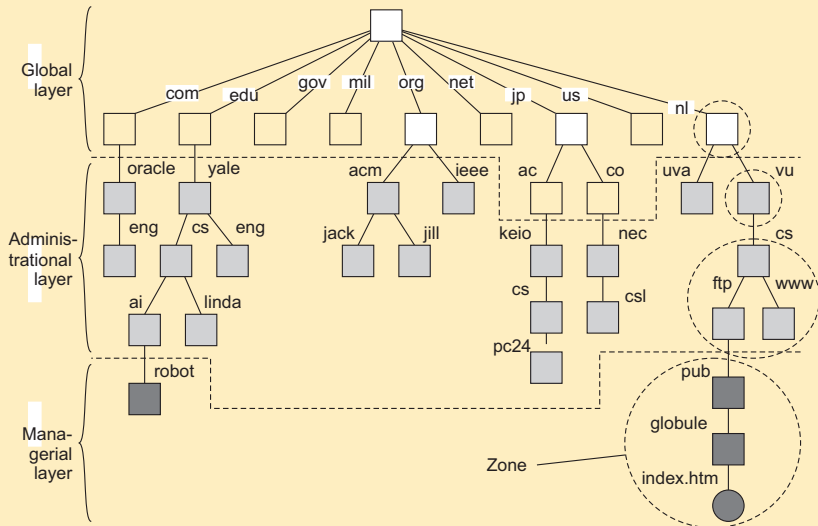
Distribute the name resolution process as well as name space management across multiple machines, by distributing nodes of the naming graph.

Distinguish three levels

- **Global level:** Consists of the high-level directory nodes. Main aspect is that these directory nodes have to be jointly managed by different administrations
- **Administrational level:** Contains mid-level directory nodes that can be grouped in such a way that each group can be assigned to a separate administration.
- **Managerial level:** Consists of low-level directory nodes within a single administration. Main issue is effectively mapping directory nodes to local name servers.

Name-space implementation

An example partitioning of the DNS name space, including network files



Name-space implementation

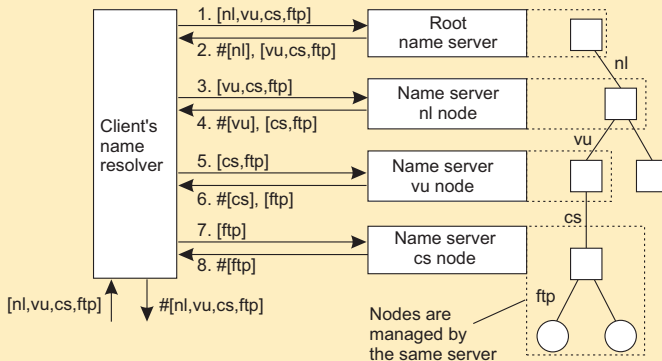
A comparison between name servers for implementing nodes in a name space

Item	Global	Administrational	Managerial
1	Worldwide	Organization	Department
2	Few	Many	Vast numbers
3	Seconds	Milliseconds	Immediate
4	Lazy	Immediate	Immediate
5	Many	None or few	None
6	Yes	Yes	Sometimes
1: Geographical scale 2: # Nodes 3: Responsiveness		4: Update propagation 5: # Replicas 6: Client-side caching?	

Iterative name resolution

Principle

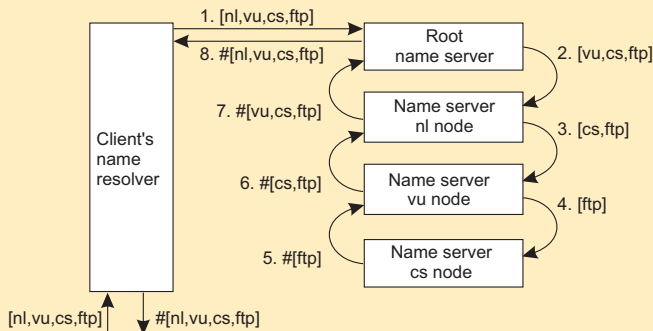
- 1 $resolve(dir, [name_1, \dots, name_K])$ sent to $Server_0$ responsible for dir
- 2 $Server_0$ resolves $resolve(dir, name_1) \rightarrow dir_1$, returning the identification (address) of $Server_1$, which stores dir_1 .
- 3 Client sends $resolve(dir_1, [name_2, \dots, name_K])$ to $Server_1$, etc.



Recursive name resolution

Principle

- 1 $resolve(dir, [name_1, \dots, name_K])$ sent to $Server_0$ responsible for dir
- 2 $Server_0$ resolves $resolve(dir, name_1) \rightarrow dir_1$, and sends $resolve(dir_1, [name_2, \dots, name_K])$ to $Server_1$, which stores dir_1 .
- 3 $Server_0$ waits for result from $Server_1$, and returns it to client.



Caching in recursive name resolution

Recursive name resolution of $[nl, vu, cs, ftp]$

Server for node	Should resolve	Looks up	Passes to child	Receives and caches	Returns to requester
<i>cs</i>	$[ftp]$	$\#[ftp]$	—	—	$\#[ftp]$
<i>vu</i>	$[cs, ftp]$	$\#[cs]$	$[ftp]$	$\#[ftp]$	$\#[cs]$ $\#[cs, ftp]$
<i>nl</i>	$[vu, cs, ftp]$	$\#[vu]$	$[cs, ftp]$	$\#[cs]$ $\#[cs, ftp]$	$\#[vu]$ $\#[vu, cs]$ $\#[vu, cs, ftp]$
<i>root</i>	$[nl, vu, cs, ftp]$	$\#[nl]$	$[vu, cs, ftp]$	$\#[vu]$ $\#[vu, cs]$ $\#[vu, cs, ftp]$	$\#[nl]$ $\#[nl, vu]$ $\#[nl, vu, cs]$ $\#[nl, vu, cs, ftp]$

Scalability issues

Size scalability

We need to ensure that servers can handle a large number of requests per time unit $\Rightarrow$ high-level servers are in big trouble.

Scalability issues

Size scalability

We need to ensure that servers can handle a large number of requests per time unit $\Rightarrow$ high-level servers are in big trouble.

Solution

Assume (at least at global and administrative level) that content of nodes hardly ever changes. We can then apply extensive replication by mapping nodes to multiple servers, and start name resolution at the nearest server.

Scalability issues

Size scalability

We need to ensure that servers can handle a large number of requests per time unit $\Rightarrow$ high-level servers are in big trouble.

Solution

Assume (at least at global and administrative level) that content of nodes hardly ever changes. We can then apply extensive replication by mapping nodes to multiple servers, and start name resolution at the nearest server.

Observation

An important attribute of many nodes is the [address](#) where the represented entity can be contacted. Replicating nodes makes large-scale traditional name servers unsuitable for locating mobile entities.

Attribute-based naming

Observation

In many cases, it is much more convenient to name, and look up entities by means of their **attributes** $\Rightarrow$ traditional **directory services** (aka **yellow pages**).

Attribute-based naming

Observation

In many cases, it is much more convenient to name, and look up entities by means of their **attributes** $\Rightarrow$ traditional **directory services** (aka **yellow pages**).

Problem

Lookup operations can be extremely expensive, as they require to match **requested attribute values**, against **actual attribute values** $\Rightarrow$ inspect **all entities** (in principle).